

Objectifs

- Choisir un concept de jeu
- Définir l'architecture globale
- Définir les technologies back/front

Cette étape est essentielle pour assurer une bonne coordination entre les parties Front-End et Back-End tout au long du développement et pour conserver une vision d'ensemble du SI qui sera mis en œuvre.

Instructions

Voir page 2

Outils pédagogiques et moyens associés

Outils de dessin et en ligne :

- Excalidraw (gratuit) en activant les bibliothèques additionnelles
- Mermaid (site web + extension VSCode)

Diagrammes divers :

- Diagramme de flux, diagramme de composants, diagramme séquentiel
- Chercher si nécessaire les différents types de diagramme UML qui peuvent aider dans ces représentations

Documentations sur l'architecture client-serveur

- <https://developer.mozilla.org/fr/>

Conseil : Utilisez Notion ou un outil similaire pour travailler de manière collaborative et partager facilement le travail réalisé

M2 INFO

Coordination Front & Back

Livrable(s)

- Lien vers le repository
- 1 (ou plus) schéma d'architecture technique
- 1 document d'intention

Points d'attention

- Cohérence de l'architecture front/back
- Les principales fonctionnalités sont identifiées
- Les choix technologiques sont justifiés
- Bonus : Le travail est présenté de façon structurée et claire

Instructions

1. Choisir un concept de jeu :

Exemples : Cardboard game, shooter, rogue-like, tower-defense, etc.

2. Définir la thématique :

Cette phase est importante car elle peut influencer sur le concept du jeu et sur les points suivants !

3. Proposer une première architecture technique :

- Technologies envisagées (back & front)
- Description du fonctionnement général (qui appelle quoi ?)
- Liste préliminaire des services principaux

4. Construire un schéma d'architecture macro :

- Interactions client/serveur
- BDD
- API Rest ou Websockets (si nécessaire)
- Tout autre élément important

5. Rédiger un document d'intention (1 à 2 pages max) :

Qui explique l'objectif du jeu, l'architecture proposée et les premiers défis techniques anticipés. Le document doit également présenter les choix technologies envisagés (stack technique, etc.).

Objectifs

- Définir les relations entre des entités métier d'un projet
- Créer un MCD
- Proposer une version du MLD

Cette étape est essentielle pour préparer les futures API et garantir la cohérence des échanges entre Front-End et Back-End.

Instructions

Voir page 2

Outils pédagogiques et moyens associés

Utiliser un outil de modélisation :

- dbdiagram.io
- Jmerise ou Looping si vous êtes old-school
- <https://dbschema.com/> (dans sa version community)

Ressources sur la modélisation de BDD :

- <https://laconsole.dev/formations/bases-de-donnees-relationnelles>
- <https://openclassrooms.com/fr/courses/6938711-modelisez-vos-bases-de-donnees>
- PDFs 1 & 2 : « Modéliser vos bases de données relationnelles »

M2 INFO

Coordination Front & Back

Livrable(s)

- Lien vers le repository
- 1 schéma MCD complet
- 1 schéma MLD cohérent avec le MCD

Points d'attention

- Les principales entités du jeu sont bien modélisées
- Les relations sont correctes et bien justifiées
- Les schémas sont lisibles, bien structurés et techniquement corrects

Instructions

Attention, contrainte projet :

→ Mettre en œuvre une BDD relationnelle (et non pas NoSQL)

1. Lister les principales entités métier du projet :

Ex: Joueur, Monstre, Carte, Score, etc.

2. Définir les relations entre ces entités :

Ex: Un joueur possède plusieurs scores, ou au contraire un joueur possède un score unique, etc.

3. Créer un MCD (Modèle conceptuel de données) :

- Entités
- Attributs
- Relations cardinalisées

4. Proposer une première version du MLD (Modèle logique de données) :

- Nom des tables
- Clés primaires & étrangères
- Type de données

Objectifs

- Mettre en œuvre la BDD du jeu
- Commencer à construire les premières routes API REST.

Cette étape est essentielle pour préparer l'interaction entre le modèle de données et l'application front-end.

Instructions

Voir page 2

Outils pédagogiques et moyens associés

Création de BDD :

- Rappels sur le SQL : [GitHub - detygon/cheatsheet-sql: Aide-mémoire SQL](#) et [GitHub - enochtangg/quick-SQL-cheatsheet: A quick reminder of all SQL queries and examples on how to use them.](#)
- Document de l'ORM sélectionné

ORM recommandés :

(cela varie énormément en fonction du backend)

- JS : Sequelize, Prisma
- Python : Django ORM, SQLAlchemy (SQLAlchemy)

Outils de test API :

- Postman, Bruno ou Insomnia
- Extension Postman sur VSCode
- Extension VSCode HTTP Client:
<https://marketplace.visualstudio.com/items/?itemName=mkloubert.vscode-http-client>
→ Super pour l'utiliser avec des "HTTP files" (cf la doc. de l'extension)

M2 INFO

Coordination Front & Back

Livrable(s)

- Lien vers le repository
- BDD avec données mocks
- Serveur d'API fonctionnelle + 2 routes

Points d'attention

- BDD opérationnelle
- Cohérence entre modèle ORM & BDD
- Respect des conventions REST

Bonus :

- Mettre en place les premiers contrôles d'erreurs simples (ex: 404 ou 500)

Instructions

1. Créer la BDD

- Créer le modèle physique de données (MPD)
- Implémenter le modèle dans un SGBD (moteur de BDD) au choix
- Utiliser directement SQL ou un ORM

2. Insérer quelques données de test

- 2 ou 3 enregistrements minimum par table
- Idéalement mettre en place un système de mock (built-in ou custom)

3. Initialiser un serveur API

- Config. un serveur avec une technologie comme Express.js, Django ou Spring par exemple (selon la stack choisie et validée lors des séance 1 & 2)

4. Créer au moins 2 routes simples sur l'API REST

- Les routes doivent interagir avec la BDD.

5. Tester les routes avec un client de test API

Objectifs

Mettre en application sur le projet fil rouge les concepts et bonnes pratiques pour valider les compétences.

Instructions

Voir page 2

Outils pédagogiques et moyens associés

- Documentation technique des technos sélectionnées
- Blogs (médium, aggregateur : stackshare.io, etc.)
- Grafikart (blog & chaine Youtube)
- Newsletter : TLDR (Webdev, etc.)
- roadmap.sh

M2 INFO

Coordination Front & Back

Livrable(s)

- Lien vers le repository
- Projet correctement initialisé
- Prototype partiellement fonctionnel
- Code source structuré et lisible
- Doc. Technique initialisée

Points d'attention

- Projet opérationnel
- Choix d'architecture et techniques
- Organisation du code et lisibilité

Instructions

Les tâches ci-dessous décrivent le processus global de conception. Ils doivent être adaptés en fonction de votre avancée sur le projet concerné.

1. Initialiser son environnement de développement

2. Définir l'architecture de projet :

- Architecture et méthodologie de conception
- Stack technique

3. Prototyper l'app :

- Définir un MVP personnel (fonctionnalités minimales attendues)
- Préparer des mockups

4. Développer un premier lot fonctionnel :

- Développer une première fonctionnalité end-to-end (ex: création d'un compte utilisateur, affichage d'un calendrier interactif, etc.)
- Structurer le code de manière modulaire et maintenable

5. Commencer la doc. technique :

- Décrire l'installation du projet (prérequis, technos, etc. dans un README.md)
- Rédiger une première version de la doc. d'architecture technique

Objectifs

Mettre en application sur le projet fil rouge les concepts et bonnes pratiques pour valider les compétences.

Instructions

Voir page 2

Outils pédagogiques et moyens associés

- Supports : présentation (fichier joint), projet de jeu
- Documentation : <https://jwt.io>
- Authentification Express.js / Django / Spring security (selon stack choisie) et document correspondante
- Outils de test API (Postman, Bruno ou Insomnia, etc.)

M2 INFO

Coordination Front & Back

Livrable(s)

- Lien vers le repository
- Authentification et sécurisation fonctionnelles sur le projet fil rouge

Points d'attention

- Auth côté back en place
- Stockage du token et transmission correcte depuis le front
- Routes API sécurisées
- Documentation mise à jour

Instructions

Les tâches ci-dessous décrivent le processus global de conception. Ils doivent être adaptés en fonction de votre avancée sur le projet concerné.

1. Implémenter une auth dans l'API de votre projet fil rouge :

- Créer une route POST /login (vérif. des id utilisateur, envoi d'un JWT, etc.)
- Créer une route POST /register

2. Sécuriser les routes sensibles de votre API :

- Implémenter un middleware pour vérifier le JWT
- Appliquer cette protection sur certaines routes

3. Adaptation du frontend :

- Ajouter un formulaire de connexion
- Stocker proprement le token récupéré à la connexion
- Utiliser ce token pour authentifier les appels API protégés

4. Gérer l'expiration et la déconnexion :

- Détecter et gérer l'expiration du token
- Proposer un bouton de déconnexion qui supprime proprement le token

Objectifs

Mise en œuvre d'une gestion propre des états d'application.

L'objectif est de renforcer la maintenabilité et la robustesse du projet.

Instructions

Voir page 2

Outils pédagogiques et moyens associés

- Doc. officielle des frameworks, frameworks de gestion d'états, etc.

M2 INFO

Coordination Front & Back

Livrable(s)

- Lien vers le repository
- Des flux opérationnels pour les différentes entités du jeu
- Un code organisé proprement
- Des comportements utilisateurs clairs

Points d'attention

- Flux fonctionnels
- Code propre et lisible
- States correctement gérés

Instructions

Objectif : Implémenter les flux clefs du jeu

1. Sélectionner un flux de données

En se basant sur les diagrammes d'architecture bâtis dans les premières séances

2. Implémenter une gestion complète d'états pour ce flux :

- Loading, success, error, etc.
- Modularisation propre (service & API séparé)

3. Afficher dynamiquement les données récupérées depuis l'API

4. Gérer les erreurs d'API de manière visible pour l'utilisateur

Dans le cadre de la gestion d'un jeu, c'est une mise en oeuvre spécifique en fonction du type de jeu

5. Organiser le code en dossiers et fichiers propres selon la logique du projet

→ Recommencer pour chaque flux identifié.

Objectifs

L'objectif est de poursuivre le développement du projet de jeu ou du projet fil rouge, en appliquant les compétences acquises :

- Communication sécurisée
- Gestion d'état
- Interaction back/front

Instructions

Voir page 2

Outils pédagogiques et moyens associés

- Codes produits sur GitHub/GitLab
- TP précédents
- Documentation officielle des stacks utilisées

M2 INFO

Coordination Front & Back

Livrable(s)

- Lien vers le repository
- Avancement significatif sur les fonctionnalités critiques

Points d'attention

- Analyse du code produit conformément à la backlog définie
- Pertinence et stabilité de la live-démo

Instructions

1. Identifier et prioriser les fonctionnalités encore manquantes :

- Auth. Terminée ?
- Affichage des données dynamiques fonctionnel ?
- Gestion d'état fonctionnelle ?

2. Définir la liste des tâches de l'après-midi :

- Mettre en place une mini backlog d'équipe ou personnelle

3. Continuer le développement :

- Correction des fonctionnalités non terminées
- Avancement sur les fonctionnalités secondaires ou bonus

4. Préparer une démo du projet (5 minutes)

- Préparer une live-démo du projet
- Prioriser les fonctionnalités développées
- Mocker les données ou les comportements si besoin

Objectifs

- Être capable de réaliser un audit technique du projet
- Identifier les principaux leviers d'amélioration avec priorisation
- Mettre en œuvre les optimisations liées concrètes

Instructions

Voir page 2

Outils pédagogiques et moyens associés

- Dev tool (Firefox ou Chrome)
- Google lighthouse (analyse frontend)
- Postman ou autre client
- Blogs tech d'acteurs importants (recherche sur stackhouse.io)

M2 INFO

Coordination Front & Back

Livrable(s)

- Lien vers le repository
- Un mini rapport d'audit (1 demi-page à 1 page) listant :
 - problèmes détectés
 - actions d'opti. menées
 - gains éventuels mesurés.

Points d'attention

- Amélioration réelle du projet
- Refactorisation visible sur le projet

Instructions

1. Compléter l'audit du projet :

- API : Temps de réponse, volume de données échangées
- Front : Web vitals, poids des pages, découpage du code
- Architecture : Organisation des fichiers, redondances, code inutile
- BDD : Evolution de la BDD (migrations), optimisations (index, etc.)

2. Identifier au moins 2 points d'optimisation concrets liés à l'audit

Exemples : Pagination d'API sur liste longue de données, chargement différé d'un composant trop lourd, correction/refactorisation d'un algorithme mal implémenté, etc.

3. Corriger ces deux points (prioriser les corrections à impact fort)

4. Valider les améliorations

Objectifs

- Réaliser un sprint de finalisation sur le jeu
- Corriger les fonctionnalités critiques
- Stabiliser le code
- Préparer le playtest.

Instructions

Voir page 2

Outils pédagogiques et moyens associés

- Grille de checklist projet (fichier joint)
- Trello / Notion / Github projects
- Canva / Powerpoint / Google slides

M2 INFO

Coordination Front & Back

Livrable(s)

- Lien vers le repository
- Projet fonctionnel et stable (ou en voie de l'être) :
 - Fonctionnalités critiques livrées et corrigées
 - UI propre et utilisable
- Support de présentation

Points d'attention

- Analyse du sprint report et du projet
- Cohérence entre les objectifs fixés et atteints

Instructions

1. Réaliser un état des lieux :

- Quelles fonctionnalités majeures sont terminées ?
- Quelles fonctionnalités sont à finaliser (ou à corriger si bug) ?
- Quels problèmes bloquants ?

2. Prioriser

- Identifier les tâches critiques avant le playtest de la séance 10
- Classer par priorité (reprendre la méthodologie MoSCoW)

3. Corriger et finaliser :

- Déboguer les fonctionnalités instables de priorité max.
- Améliorer la finition UI/UX si possible

4. Préparation de la démo :

- Préparer un support (quelques slides) de présentation
- S'entraîner à présenter l'architecture, les choix techniques et le concept du jeu

Objectifs

- Cet ultime TP vise à appliquer l'ensemble des compétences acquises (coordination front/back, sécurisation, optimisation, etc.) au projet fil rouge.
- L'objectif est d'avoir un socle technique solide pour le rendu et une soutenance.

Instructions

Voir page 2

Outils pédagogiques et moyens associés

- Projet réalisé durant le module
- Bonnes pratiques, recommandations et autres supports fournis précédemment.

M2 INFO

Coordination Front & Back

Livrable(s)

- Github du projet fil rouge avec preuves de consolidation
- Git propre pour une analyse extérieure
- Rapport d'analyse critique

Points d'attention

- Git propre pour une analyse extérieure
- Respect des objectifs fixés et des concepts vus lors des séances/TP

Instructions

À réaliser sur le projet fil rouge.

1. Réaliser une analyse critique du projet fil rouge :

- Quels sont les points forts et les points faibles ?
- Quelles parties peuvent ou doivent être refactorisées ?
- Quels apports du module pour le projet fil rouge ?

2. Définir une feuille de route "post-module" :

- Lister les risques techniques ou fonctionnels
- Proposer un plan d'action simple et priorisé

3. Mettre en oeuvre le plan d'action :

- Traiter les tâches priorisées en vue de la réussite du rendu/soutenance
- Traiter les tâches critiques qui représentent un danger pour les objectifs fixés